Cryptography

CSUMB

Sam Ogden

OSTEP 56

Updated 2026-04-29 09:41

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Lecture Objectives

- After this lecture, you should be able to:
- Talk about the two main kinds of cryptography
- Discuss how we use cryptography in operating systems and computer science
- Know to never, ever roll your own (at least until you do much too much math)

Major Goals of Cryptography

Confidentiality

- Hide the information from prying eyes
- Encryption

Authenticity

- Ensure the data came from who it says it came from
- Signing

Integrity

- Ensure the data is correct
- Checksums

Confidentiality

Ensuring data is secret

Two main kinds of Encryption

- Symmetric Key encryption
  - Both the sender and receiver use the same key
  - Examples: DES, AES, Blowfish
- Asymmetric key encryption
  - The sender and the receiver use two different keys, the public key and a private key
  - Examples: RSA, ECC

Shared-Key Encryption (Symmetric)

We are given:

- An algorithm (aka `cipher`), E
- Some data (aka `plaintext`), P
- One secret (aka `key`), K

$$C = E(P, K)$$
$$P = D(C, K)$$

We output:

- Encrypted data (aka `ciphertext`), C

Public-Key Encryption (Asymmetric)

We are given:

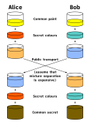
- An algorithm (aka `cipher`), E
- Some data (aka `plaintext`), P
- Two keys
  - A **public** K_{public}
  - A **secret** $K_{private}$

$$C = E(P, K_{public})$$
$$P = E(C, K_{private})$$

Trade-offs

- Public Key encryption is expensive
  - Long keys (thousands of bits)
  - Lots of calculations (huge exponents)
- Shared Key encryption is onerous
  - Either you have many unique keys, or you reduce privacy greatly
- Often we use a mix of both!

Diffie-Hellman Key Exchange



The Cryptography’s Benefit Relies Entirely On The Secrecy Of The Key

Integrity

Verifying data is right

Integrity: Core Idea

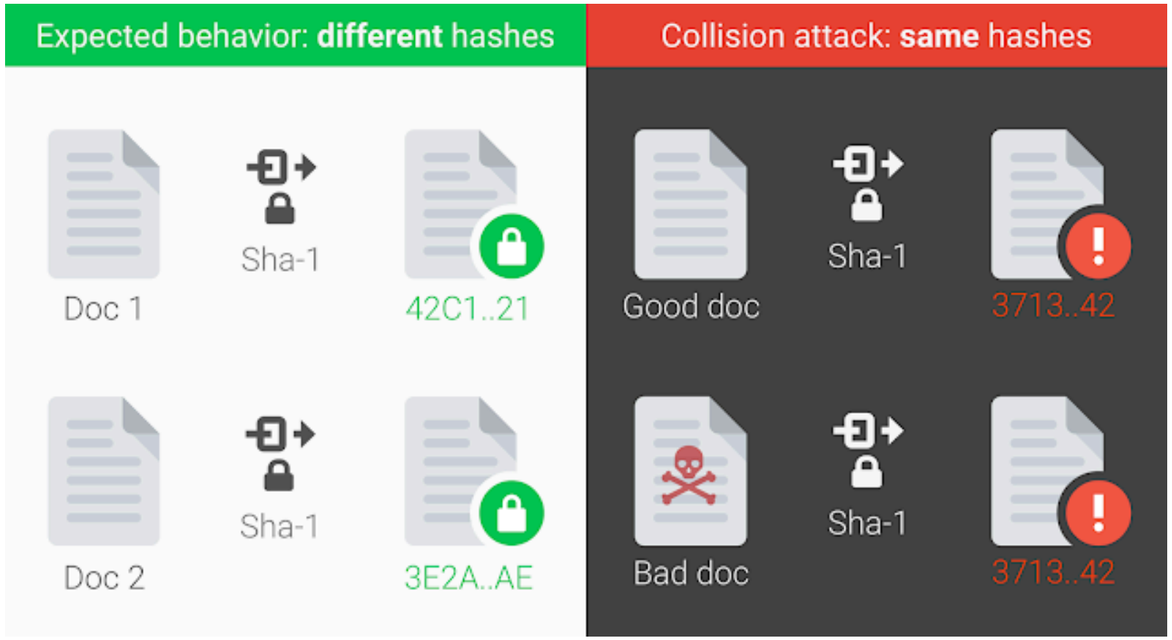
- Include a piece of information to ensure that a file is unaltered
- We accomplish this by adding in an extra piece of information: a hash
- Hashes are one-way functions that represent data in a compact form

Features of Cryptographic Hashes

- **Collision Resistance**: Collisions are computationally infeasible
- **Input sensitivity**: Any input change results in large output change
  - Roughly 50% of bits in output should change for any single input bit change
- **One-way**: Computationally infeasible to infer input properties

Computationally Infeasible

Hashes: Collisions



Hashes: SHA-1 Collision

- Google made a SHA-1 collision happen in 2017
- It was previously considered secure
- Some numbers:
  - Nine quintillion (9,223,372,036,854,775,808) SHA1 computations in total
  - 6,500 years of CPU computation to complete the attack first phase
  - 110 years of GPU computation to complete the second phase

Hashes: Small Changes to Big Changes

```
$ sha1 1 2
SHA1 (1) = 356a192b7913b04c54574d18c28d46e6395428ab
SHA1 (2) = da4b9237bacccdf19c0760cab7aec4a8359010b0
```

We change one bit in the input and it changes everything in the output

Hashes: One-way

```
❯ wc -c 56-cryptography.qmd && sha1 56-cryptography.qmd
    4649 56-cryptography.qmd
SHA1 (56-cryptography.qmd) = fb1c5078ca0db9333dd868a4c7ae22a5eabdfe33
```

Notice that we went from a 4649 byte file to a short string!

Authentication

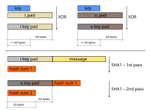
Verifying we are us

Core Idea: Authentication

Our goal is to prove that we are us, and that we did a thing

- We can do this by sharing something **only** we would know
- We can sign the file with our **private** key, which is known only to us
- Recipient can then verify with our **public** key
  - Remember that our public and private keys are two sides of the same algorithm

Example of Authentication: HMAC



Breaking Crytography

Common Ways to crack Cryptography

Computational threats

- Computational threats
  - Brute-forcing keys
  - Side-channel attacks
  - Quantum Computers

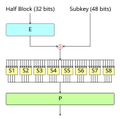
Operational Issues

- Operational Issues
  - Social Engineering
  - Backdoors

Design Flaws

- Software errors
- Bad cryptosystem design

Example of a flawed system: DES



Cryptography in Operating Systems

What the OS should think about

- How to provide **entropy**
- How to protect the user against **us**
- How to provide **at-rest data encryption**
- How to safely authenticate users across systems